

Deploying an AI Model Locally Without Internet

Offline Deployment of a 27B Cybersecurity-Focused Language Model Using llama.cpp

Detail	Description
Model	Qwen3.8-27B-Uncensored-Cyber (GGUF format)
Inference Engine	llama.cpp (CUDA build)
Environment	Kaggle GPU Runtime — 2 × NVIDIA Tesla T4
Objective	Local / offline inference without an online AI service
Date	September 20, 2026

Author:
Azam Abbas & Awaiz Ahmed

Table of Contents

1. Introduction.....	3
2. Objectives	3
3. Kaggle Hardware / GPU Specifications.....	3
4. llama.cpp Setup.....	4
5. Model Download and Verification	6
6. Dual-GPU Configuration	7
7. Problems Faced and Solutions	8
8. Online Inference.....	8
9. Offline Inference.....	9
10. Test Results.....	11
11. Cloud vs. Local Execution.....	12
12. Security and Privacy	13
13. What I Learned	13
14. Conclusion	13
15. Screenshots	13

1. Introduction

The purpose of this assignment was to deploy a 27B cybersecurity-focused AI model using llama.cpp and run inference without relying on an online AI service. The model used was Qwen3.8-27B-Uncensored-Cyber, stored in GGUF format.

2. Objectives

- Deploy a large language model using llama.cpp.
- Utilize available GPU resources for inference.
- Load the model from local storage.
- Test the model with cybersecurity-related questions.
- Verify that inference works when Internet access is disabled.
- Understand the difference between cloud infrastructure and local model execution.

3. Kaggle Hardware / GPU Specifications

The deployment was performed in a Kaggle GPU runtime.

Component	Specification
GPUs	2 × NVIDIA Tesla T4
GPU memory	~15 GB per GPU
CUDA	13.0
Driver	580.159.04
Operating System	Linux x86_64
Compiler	GNU 11.4.0

Both GPUs were successfully detected by the system and made available to llama.cpp.

32]:

`!nvidia-smi`

Sun Sep 20 09:02:45 2026

NVIDIA-SMI 580.159.04			Driver Version: 580.159.04			CUDA Version: 13.0		
GPU	Name		Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC
Fan	Temp	Perf	Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	MIG M.
=====								
0	Tesla T4		Off	00000000:00:04.0	Off			0
N/A	74C	P8	13W / 70W		0MiB / 15360MiB	0%	Default	N/A
=====								
1	Tesla T4		Off	00000000:00:05.0	Off			0
N/A	74C	P8	12W / 70W		0MiB / 15360MiB	0%	Default	N/A
=====								
Processes:								
GPU	GI	CI	PID	Type	Process name	GPU Memory		
	ID	ID				Usage		
=====								

Figure 1: Kaggle environment showing two NVIDIA Tesla T4 GPUs.

4. llama.cpp Setup

llama.cpp was cloned from its GitHub repository and compiled with CUDA support. The build was configured using:

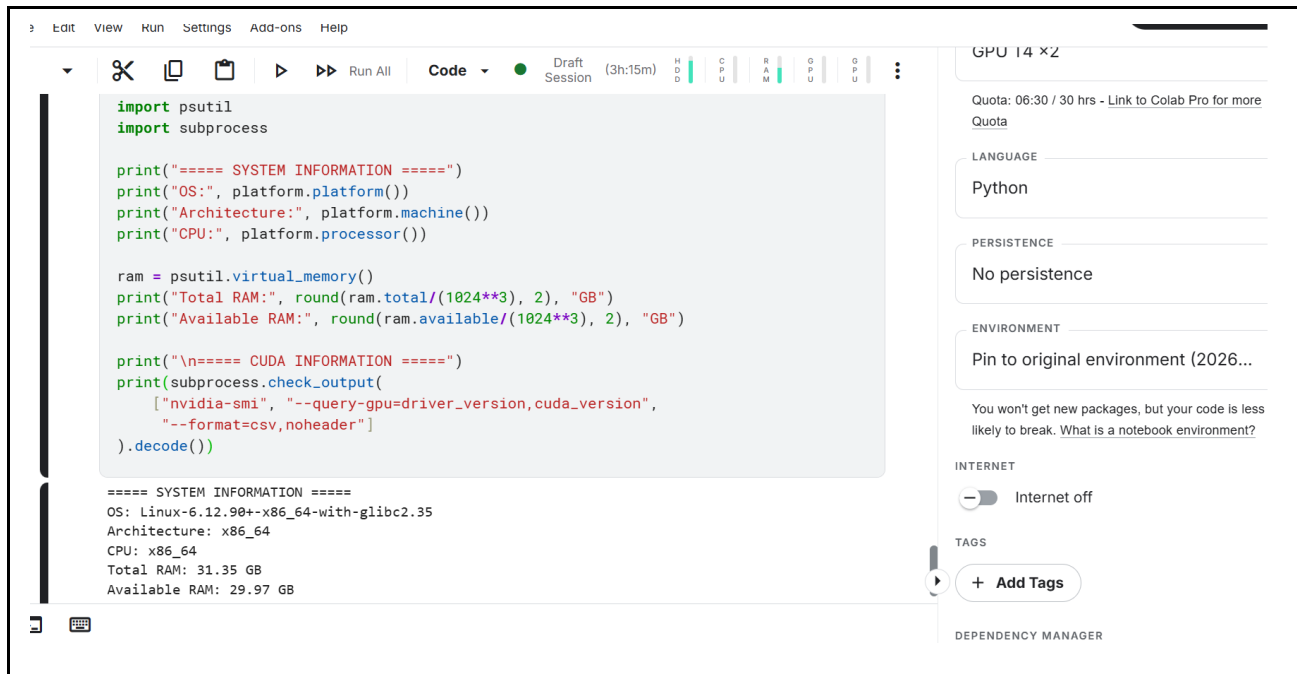
```
cmake -B build \
-DGGML_CUDA=ON \
-DGGML_CUDA_NO_VMM=ON \
-DCMAKE_CUDA_COMPILER=/usr/local/cuda/bin/nvcc \
-DCUDAToolkit_ROOT=/usr/local/cuda
```

The resulting version was:

0.4.1-dev

build 11061

commit b23efaa2e



The screenshot shows a Google Colab notebook interface. The main code cell contains Python code that imports `psutil` and `subprocess`, then prints system information (OS, Architecture, CPU, Total RAM, Available RAM) and CUDA information (using `nvidia-smi`). The output of the code is visible below the code cell, showing the system details. On the right sidebar, the GPU is set to 'GPU 14 x2', the language is 'Python', and persistence is set to 'No persistence'. The internet is turned off.

```
import psutil
import subprocess

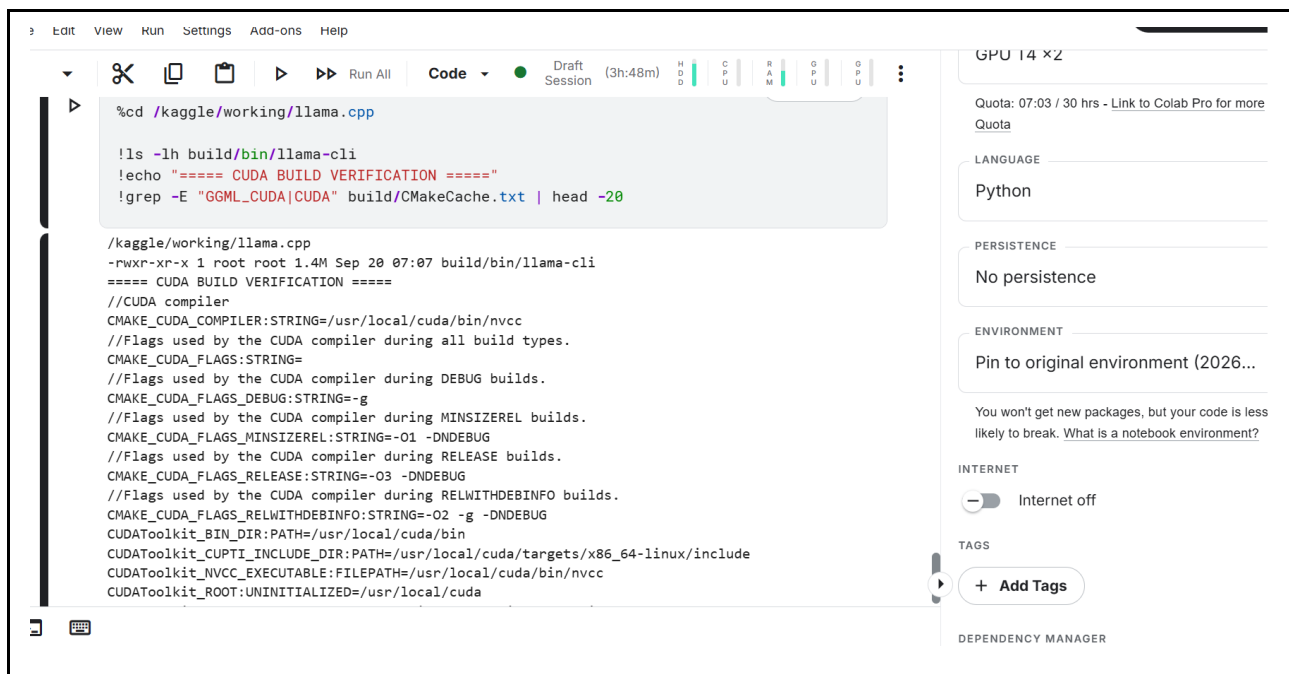
print("==== SYSTEM INFORMATION =====")
print("OS:", platform.platform())
print("Architecture:", platform.machine())
print("CPU:", platform.processor())

ram = psutil.virtual_memory()
print("Total RAM:", round(ram.total/(1024**3), 2), "GB")
print("Available RAM:", round(ram.available/(1024**3), 2), "GB")

print("\n==== CUDA INFORMATION =====")
print(subprocess.check_output(
    ["nvidia-smi", "--query-gpu=driver_version,cuda_version",
     "--format=csv,noheader"]
).decode())

==== SYSTEM INFORMATION =====
OS: Linux-6.12.90+-x86_64-with-glibc2.35
Architecture: x86_64
CPU: x86_64
Total RAM: 31.35 GB
Available RAM: 29.97 GB
```

Figure 2: CUDA and system environment verification.

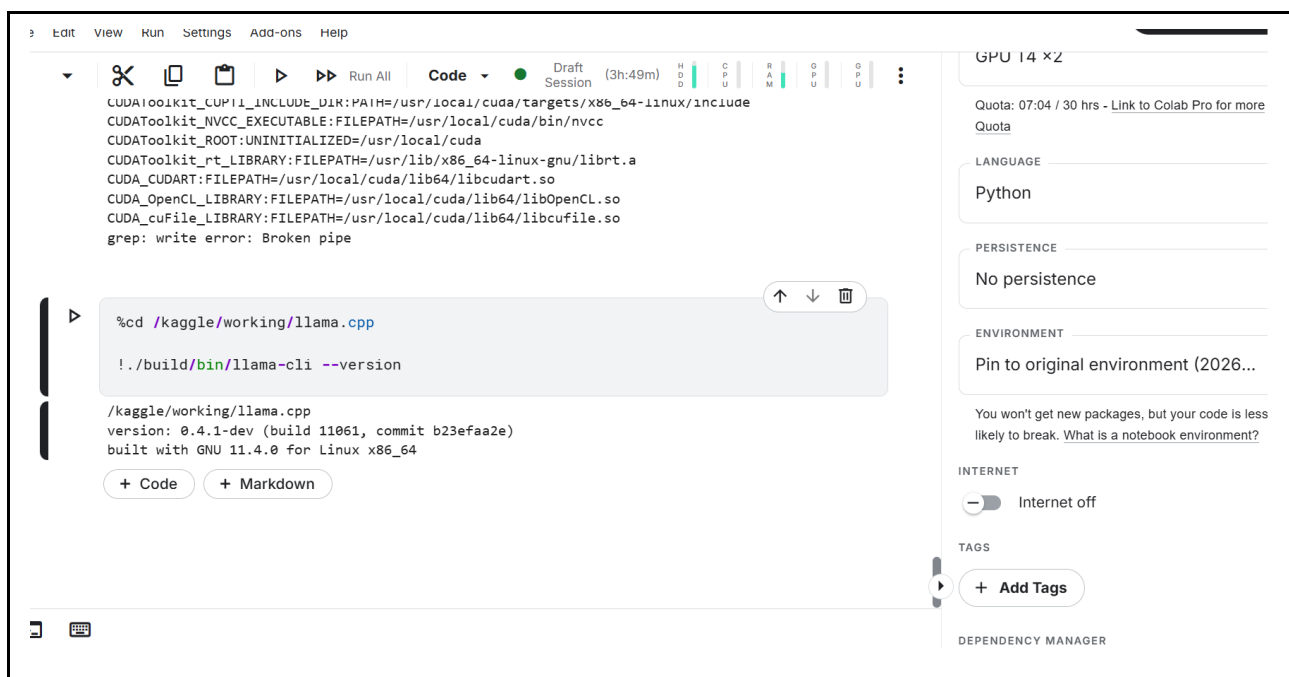


```
%cd /kaggle/working/llama.cpp

!ls -lh build/bin/llama-cli
!echo "==== CUDA BUILD VERIFICATION ====="
!grep -E "GGML_CUDA|CUDA" build/CMakeCache.txt | head -20

/kaggle/working/llama.cpp
-rwxr-xr-x 1 root root 1.4M Sep 20 07:07 build/bin/llama-cli
==== CUDA BUILD VERIFICATION =====
//CUDA compiler
CMAKE_CUDA_COMPILER:STRING=/usr/local/cuda/bin/nvcc
//Flags used by the CUDA compiler during all build types.
CMAKE_CUDA_FLAGS:STRING=
//Flags used by the CUDA compiler during DEBUG builds.
CMAKE_CUDA_FLAGS_DEBUG:STRING=-g
//Flags used by the CUDA compiler during MINSIZEREL builds.
CMAKE_CUDA_FLAGS_MINSIZEREL:STRING=-O1 -DNDEBUG
//Flags used by the CUDA compiler during RELEASE builds.
CMAKE_CUDA_FLAGS_RELEASE:STRING=-O3 -DNDEBUG
//Flags used by the CUDA compiler during RELWITHDEBINFO builds.
CMAKE_CUDA_FLAGS_RELWITHDEBINFO:STRING=-O2 -g -DNDEBUG
CUDAToolkit_BIN_DIR:PATH=/usr/local/cuda/bin
CUDAToolkit_CUPTI_INCLUDE_DIR:PATH=/usr/local/cuda/targets/x86_64-linux/include
CUDAToolkit_NVCC_EXECUTABLE:FILEPATH=/usr/local/cuda/bin/nvcc
CUDAToolkit_ROOT:UNINITIALIZED=/usr/local/cuda
```

Figure 3: Successful compilation of llama.cpp with CUDA support.



```
%cd /kaggle/working/llama.cpp

!./build/bin/llama-cli --version

/kaggle/working/llama.cpp
version: 0.4.1-dev (build 11061, commit b23efaa2e)
built with GNU 11.4.0 for Linux x86_64
```

Figure 4: llama.cpp version and build verification.

5. Model Download and Verification

The model was downloaded from the specified Hugging Face repository in GGUF format.

Model

Qwen3.8-27B-Uncensored-Cyber-IQ4_XS-imatrix-fromq8.gguf

Size

14.96 GiB

The model file was successfully verified in:

/kaggle/working/models/

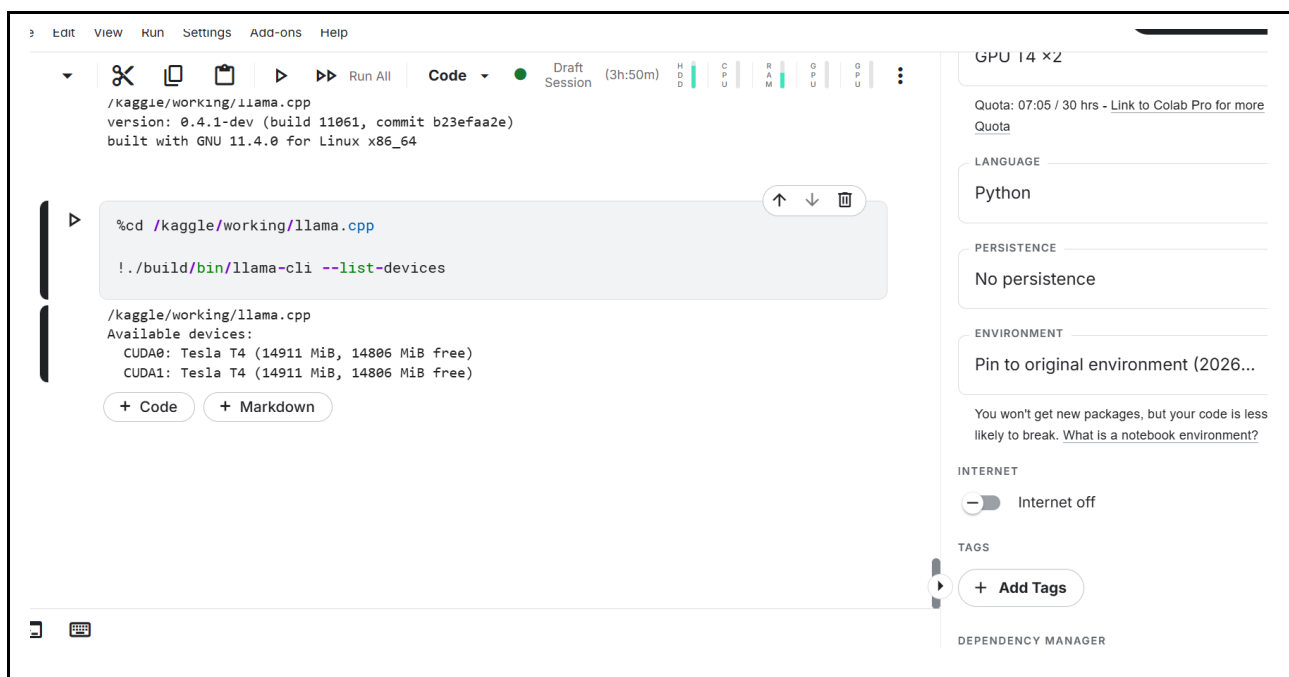


Figure 5: Detection of both Tesla T4 GPUs by llama.cpp.

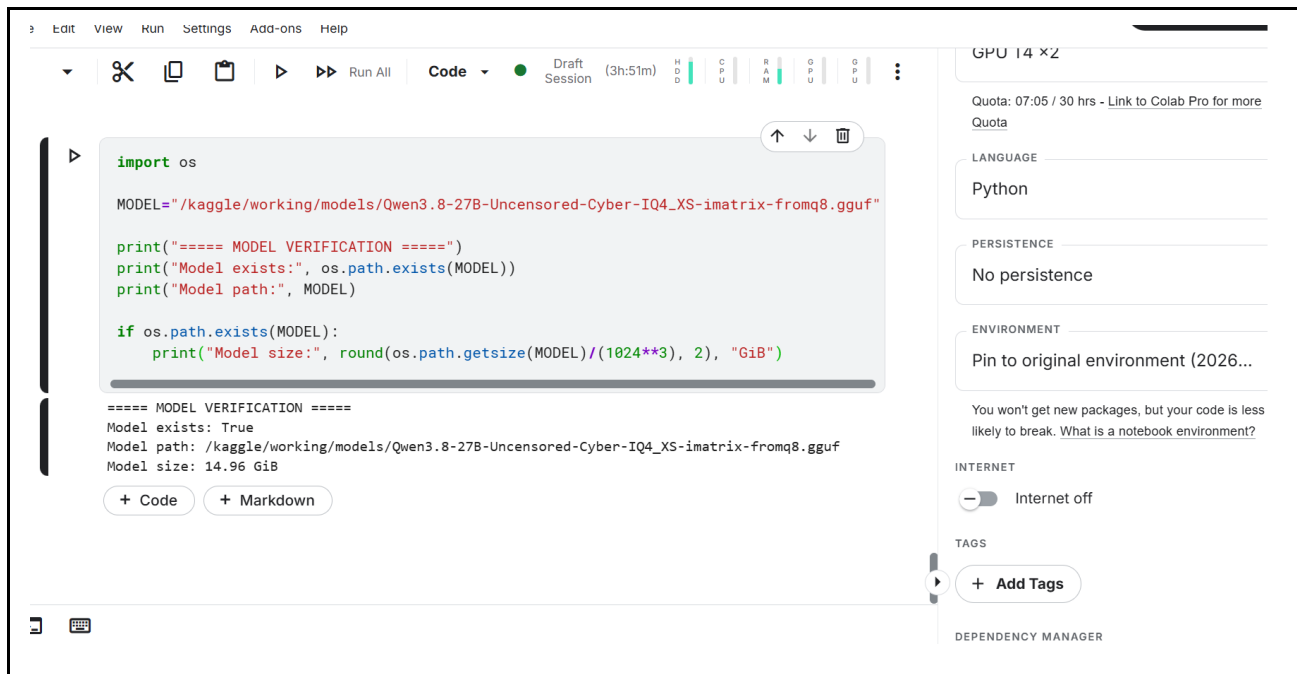


Figure 6: Download and verification of the Qwen3.8-27B GGUF model.

6. Dual-GPU Configuration

Both Tesla T4 GPUs were used for inference. llama.cpp was configured with:

```
--device CUDA0,CUDA1
--split-mode layer
--tensor-split 1,1
-ngl 99
```

This allowed the model layers to be distributed across the two available GPUs.

7. Problems Faced and Solutions

- **Model filename / 404 error:** The initial filename did not match the repository filename. The correct GGUF filename was identified and downloaded.
- **Large model size:** The 27B model required approximately 15 GB of storage, so storage usage had to be monitored.
- **Long generation / waiting:** The model uses a reasoning process and can take time to generate an answer. A generation limit of 256 tokens was used for practical testing.
- **Interactive prompt:** After answering, llama-cli displayed ">", which means it was waiting for another prompt rather than being stuck.
- **Offline testing:** After disabling Kaggle Internet access, the locally stored model and llama.cpp executable remained available and successfully performed inference.
- **Kaggle session not starting:** The Kaggle GPU notebook session sometimes failed to start or remained stuck on "Starting session..." because the requested dual-T4 accelerator was busy or the GPU quota was close to its limit. **Solution:** The session was stopped and restarted, the GPU accelerator was re-selected from Settings, and the notebook was re-run after a short wait. Checking the remaining weekly GPU quota beforehand and retrying during a less busy period resolved the issue.
- **Stuck at ">" when generating a response:** After typing a prompt into llama-cli's interactive mode, the terminal appeared to freeze at the ">" line with no new output for a while, which looked like the process had hung. **Solution:** This was not a hang — the 27B model was still processing the prompt and generating tokens in the background. Checking nvidia-smi in a separate cell confirmed GPU-Util was active during this time. Waiting for generation to finish, and setting a token limit (256 tokens) for faster, more predictable responses during testing, resolved the apparent freeze.

8. Online Inference

Before disabling Internet access, the model was successfully tested with a cybersecurity question about the CIA triad.

The model generated an appropriate answer explaining:

- Confidentiality
- Integrity
- Availability

This confirmed that the model and inference engine were functioning correctly.



Figure 7: Successful online inference using the cybersecurity model.

9. Offline Inference

The most important test was performed after Kaggle Internet access was disabled. The connectivity test reported:

Internet access: DISABLED/UNREACHABLE

The model was then loaded directly from the local filesystem and successfully answered the CIA triad question. This demonstrated that model inference did not require an active Internet connection or an online AI API.

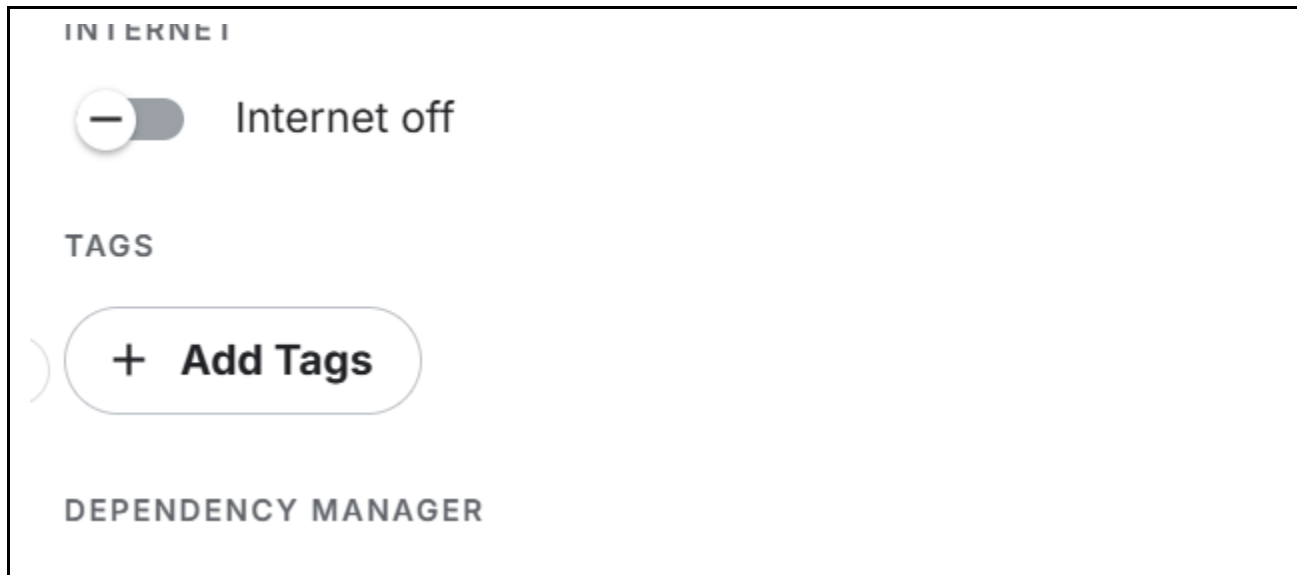


Figure 8: Kaggle Internet access disabled.

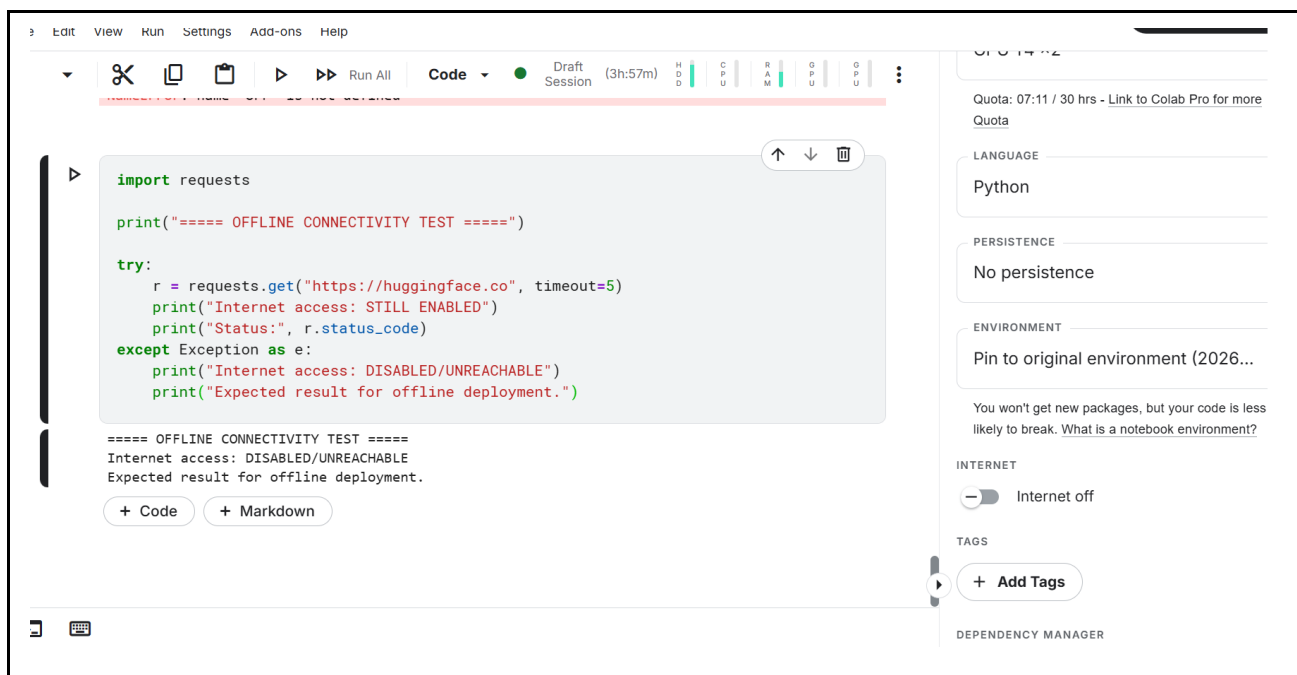


Figure 9: Verification that Internet access was disabled.



Figure 10: Successful offline inference using the locally stored 27B model.

10. Test Results

The successful offline test produced approximately:

Prompt processing: 94.5 tokens/second

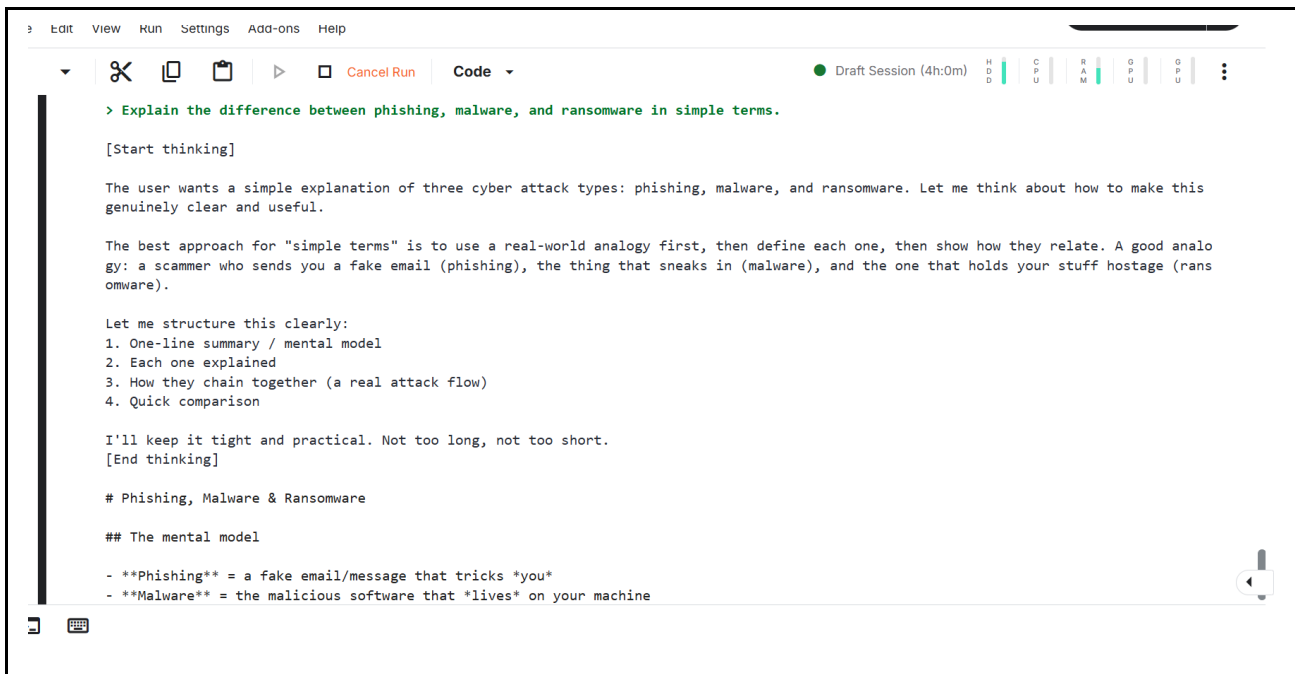
Generation: 15.2 tokens/second

The model successfully answered the following prompt:

“What is the CIA triad in cybersecurity?”

The response correctly described Confidentiality, Integrity, and Availability.

Additional cybersecurity prompts can be used to further demonstrate the model's functionality.



The screenshot shows a code editor interface with a menu bar (Edit, View, Run, Settings, Add-ons, Help) and a toolbar with icons for undo, redo, copy, paste, and a 'Cancel Run' button. The main text area contains a prompt and a generated response. The prompt is: '> Explain the difference between phishing, malware, and ransomware in simple terms.' The response starts with '[Start thinking]' and provides a detailed explanation of the three cyber attack types, including a real-world analogy and a structured list of points. The response ends with '[End thinking]' and a summary of the three attack types.

```
> Explain the difference between phishing, malware, and ransomware in simple terms.

[Start thinking]

The user wants a simple explanation of three cyber attack types: phishing, malware, and ransomware. Let me think about how to make this genuinely clear and useful.

The best approach for "simple terms" is to use a real-world analogy first, then define each one, then show how they relate. A good analogy: a scammer who sends you a fake email (phishing), the thing that sneaks in (malware), and the one that holds your stuff hostage (ransomware).

Let me structure this clearly:
1. One-line summary / mental model
2. Each one explained
3. How they chain together (a real attack flow)
4. Quick comparison

I'll keep it tight and practical. Not too long, not too short.
[End thinking]

# Phishing, Malware & Ransomware

## The mental model

- **Phishing** = a fake email/message that tricks *you*
- **Malware** = the malicious software that *lives* on your machine
```

Figure 11: Cybersecurity test prompt and generated response.

[Prompt: 97.8 t/s | Generation: 15.3 t/s]

Figure 12: Inference performance showing approximately 94.5 tokens/s prompt processing and 15.2 tokens/s generation.

11. Cloud vs. Local Execution

The experiment demonstrates an important distinction between cloud infrastructure and cloud AI services.

Kaggle provided the GPU computing environment, so the physical hardware was cloud-hosted. However, during the final test:

- The model weights were stored in the runtime.
- llama.cpp was executed locally within the runtime.
- No online AI inference API was used.
- Internet access was disabled.

Therefore, the experiment demonstrates local/offline model inference within a cloud-provided compute environment rather than inference on a personal physical laptop.

12. Security and Privacy

Running a model locally/offline can provide privacy advantages because sensitive prompts and data do not need to be sent to an external AI inference service.

For cybersecurity applications, this can be useful when working with:

- Confidential information
- Internal security documentation
- Sensitive logs
- Private organizational data

However, security also depends on protecting the machine, model files, input data, and generated outputs.

13. What I Learned

- How to verify GPU and CUDA availability.
- How to compile llama.cpp with CUDA support.
- How GGUF models are deployed locally.
- How to distribute a large model across multiple GPUs.
- How to troubleshoot model-loading and inference problems.
- How to verify that a model can operate without Internet access.
- The difference between using a cloud AI service and running an AI model locally within a cloud compute environment.

14. Conclusion

The Qwen3.8-27B cybersecurity model was successfully deployed using llama.cpp with two NVIDIA Tesla T4 GPUs. The model was loaded from local storage and successfully generated cybersecurity responses. Most importantly, the model continued to operate after Internet access was disabled, demonstrating that the inference process itself did not depend on an online AI service.

15. Screenshots

Figures 1 through 12, embedded throughout this report, document each stage of the deployment process, from GPU detection and llama.cpp compilation through online inference, disabling Internet access, and the final offline inference test.